A PROJECT REPORT ON

AUTONOMOUS CAR USING ROS2

Submitted in partial fulfillment of requirements for the award of the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

Submitted by:

K. Sai Pawan	23095A3308
P. Manoj	22091A3328
B. Jaya Naveen Singh	23095A3303
B. Sai Charan	23095A3309

Under the Esteemed Guidance of

Ms. Khadija Jabeen M.Tech, (Ph. D)

Assistant Professor, Dept. of CSE (AI & ML)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

RAJEEV GANDHI MEMORIAL COLLEGE OF ENGINEERING & TECHNOLOGY
(AUTONOMOUS)

AFFILIATED TO J.N.T UNIVERSITY ANANTAPUR. ACCREDITED BY NBA (TIER-1) & NAAC OF UGC. NEW DELHI, WITH A+ GRADE

RECOGNIZED UGC-DDU KAUSHAL KENDRA
NANDYAL-518501, (Estd-1995)

YEAR: 2025-2026

Rajeev Gandhi Memorial College of Engineering &Technology

(AUTONOMOUS)

AFFILIATED TO J.N.T UNIVERSITY ANANTAPUR. ACCREDITED BY NBA (TIER-1) & NAAC OF UGC. NEW DELHI, WITH A+ GRADE

RECOGNIZED UGC-DDU KAUSHAL KENDRA
NANDYAL-518501, (Estd-1995)

(ESTD – 1995)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

CERTIFICATE

This is to certify that **K. Sai Pawan** (*23095A3308*), **P. Manoj** (*22091A3328*), **B. Jaya Naveen Singh** (*23095A3303*) and **B. Sai Charan** (*23095A3309*) of IV- B. Tech II- semester, have carried out the major project work entitled "**AUTONOMOUS CAR USING ROS2**" under the supervision and guidance of **Ms. K. Jabeen,** Assistant Professor, CSE (AI & ML) Department, in partial fulfillment of the requirements for the award of Degree of **Bachelor of Technology** in **Computer Science and Engineering (Artificial Intelligence & Machine Learning)** from **Rajeev Gandhi Memorial College of Engineering & Technology (Autonomous)**, Nandyal is a bonafide record of the work done by them during 2025-2026.

Project Guide

Ms. K. Jabeen M. Tech, (Ph. D)
Assistant Professor
Dept. of CSE (AI & ML)
Place: Nandyal

Date:

Head of the Department

Dr. G. Kishor Kumar M.Tech, Ph.D.
Professor & HOD
Dept. of CSE (AI&ML)

External Examiner

Candidate's Declaration

We hereby declare that that the work done in this project entitled **"AUTONOMOUS CAR USING ROS2"** submitted towards completion of major project in IV Year II Semester of B. Tech (CSE(AI & ML)) at the **Rajeev Gandhi Memorial College of Engineering & Technology**, Nandyal. It is an authentic record our original work done under the esteemed guidance of **Ms. Khadija Jabeen,** Assistant Professor, department of **COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)**, RGMCET, Nandyal.

We have not submitted the matter embodied in this report for the award of any other Degree in any other institutions for the academic year 2025-2026.

By

K. Sai Pawan

(23095A3308)

P. Manoj
(22091A3328)

B. Jaya Naveen Singh

(23095A3303)

B. Sai Charan

(23095A3309)

Dept. of CSE(AI & ML), RGMCET.

Place: Nandyal

Date:

ACKNOWLEDGEMENT

We manifest our heartier thankfulness pertaining to your contentment over our project guide **Ms. Khadija Jabeen,** Assistant Professor of Computer Science Engineering (Artificial Intelligence & Machine Learning) department, with whose adroit concomitance the excellence has been exemplified in bringing out this project to work with artistry.

We express our gratitude to **Dr. G. Kishor Kumar garu,** Head of the Department of Computer Science Engineering (Artificial Intelligence & Machine Learning) department, all teaching and non-teaching staff of the Computer Science Engineering department of Rajeev Gandhi memorial College of Engineering and Technology for providing continuous encouragement and cooperation at various steps of our project.

Involuntarily, we are perspicuous to divulge our sincere gratefulness to our Principal, **Dr. T. Jaya Chandra Prasad garu,** who has been observed posing valiance in abundance towards our individuality to acknowledge our project work tangentially.

At the outset we thank our honourable **Chairman Dr. M. Santhi Ramudu garu,** for providing us with exceptional faculty and moral support throughout the course.

Finally we extend our sincere thanks to all the **Staff Members** of CSE-(AI&ML) Department who have co-operated and encouraged us in making our project successful.

Whatever one does, whatever one achieves, the first credit goes to the **Parents,** be it not for their love and affection, nothing would have been responsible. We see in every good that happens to us their love and blessings.

BY

K. Sai Pawan
(23095A3308)

P. Manoj
(22091A3328)

B. Jaya Naveen Singh
(23095A3303)

B. Sai Charan
(23095A3309)

ABSTRACT

The transportation industry experiences rapid change because self-driving vehicle technology enables vehicles to operate without needing continuous human control. The project "Automation Car Using ROS2" develops an autonomous vehicle through its use of a Robot Operating System (ROS) which provides a strong and adaptable framework for building autonomous systems. The system uses multiple sensors including LiDAR and ultrasonic sensors and camera modules and GPS to create precise environmental understanding and continuous data collection capabilities.

The sensors work as a team to create systems which detect obstacles and identify lanes and plan paths and navigate safely in autonomous mode. The Robot Operating System (ROS) functions as a middleware platform which enables developers to create robot software through its collection of tools and libraries and hardware abstraction and standardized module communication.

The proposed vehicle uses SLAM (Simultaneous Localization and Mapping) techniques to create real-time maps of its environment while it finds its exact position in that space. The vehicle uses sensor fusion together with intelligent decision-making algorithms to identify objects and steer clear of obstacles and change its driving route in real time. The system includes automated navigation methods which select the best routes for travel while reducing overall travel duration. The experimental results show that the system achieves efficient navigation with a major decrease in collision risks and an increase in path optimization precision. The system demonstrates dependable performance in both simulated environments and actual controlled field tests. The project develops a research and development solution which supports autonomous vehicle studies through its scalable and adaptable and affordable design while advancing intelligent transportation systems and robotic automation research.

Keywords: Autonomous Vehicle, Path planning, SLAM, Robot Operating System (ROS), Sensor Fusion, Attribute-based, data integrity.

CONTENTS

LIST OF ABBREVIATIONS

ROS2	-	Robot Operating System 2
SLAM	-	Simultaneous Localization and Mapping
LiDAR	-	Light Detection and Ranging
GPS	-	Global Positioning System
IMU	-	Inertial Measurement Unit
AI	-	Artificial Intelligence
ML	-	Machine Learning
CV	-	Computer Vision
PID	-	Proportional Integral Derivative (Controller)
RRT	-	Rapidly-exploring Random Tree
A*	-	A-Star Algorithm
EKF	-	Extended Kalman Filter
UKF	-	Unscented Kalman Filter
QoS	-	Quality of Service
DDS	-	Data Distribution Service
API	-	Application Programming Interface
GUI	-	Graphical User Interface
UAV	-	Unmanned Aerial Vehicle

LIST OF FIGURES

LIST OF TABLES

CHAPTER - 1

INTRODUCTION TO AUTONOMOUS CAR

1.1 Overview of Autonomous Car

The Robot Operating System 2 enables autonomous vehicle systems to operate without human assistance through their ability to perceive their surroundings and process information and move through space. The system provides a framework that enables integration of sensors and control systems and navigation systems without the need to manage hardware components at their basic level. Perception and mapping and navigation together form three fundamental elements which define autonomous navigation systems. Perception enables the vehicle to detect its environment through the use of 2D LiDAR sensors as its sensing equipment. The sensors perform continuous environmental scanning which produces distance information that enables the system to identify obstacles and comprehend the layout of its surroundings. Researchers use Simultaneous Localization and Mapping techniques to achieve both localization and mapping functions. This system enables the vehicle to create a map of the unknown territory while it tracks its location on that map. The Adaptive Monte Carlo Localization (AMCL) algorithm uses probabilistic techniques to enhance position determination accuracy. The navigation system enables the vehicle to travel from its initial point to its final destination. The Navigation2 (Nav2) stack in ROS2 performs both path planning and obstacle detection through its navigation capabilities. The system uses the generated map to find the best routes which it maintains by tracking incoming obstacles. The implementation of autonomous car systems brings multiple benefits which include decreased need for human operators, better operational efficiency, and greater protection against risks. The systems enable vehicles to navigate through intricate environments because they can adjust to sudden alterations in their surroundings. The systems experience operational difficulties because they depend on sensors which have restricted capabilities and they need to manage unpredictable environmental conditions and their need for human operators.

1.2. Characteristics of Autonomous Car using ROS2

The Robot Operating System 2 autonomous vehicle systems demonstrate their primary ability to operate autonomously. The system achieves complete autonomous operation through its combination of sensing capabilities and computational strength and control systems. The system

requires these specific components as its fundamental operational elements essential to its functioning:

- **Modular Architecture:** The system is divided into independent modules (nodes) such as perception, localization, and navigation. The modules operate separately while they use ROS2 topics and services to communicate which allows the system to be updated and operate in different ways.

- **Real-Time Communication:** The ROS2 system enables quick and dependable data transmission between its different system components. The system achieves immediate environmental response capabilities through its real-time processing of sensor data and control commands and navigation updates.

- **Sensor Integration:** Autonomous systems use 2D LiDAR sensors to perform continuous environmental scanning. The sensors provide autonomous systems with accurate distance measurements and obstacle detection which enables safe navigation.

- **Simultaneous Localization and Mapping Capability:** The system uses Simultaneous Localization and Mapping technology to create environmental maps while determining its position on those maps which enables operation in areas without GPS access.

- **Dynamic Path Planning:** The Navigation2 stack enables the robot to determine optimal routes which it adjusts in response to obstacle detection to maintain safe and effective movement.

- **Scalability and Flexibility:** The ROS2 framework allows the system to be extended with additional sensors, algorithms, or hardware components without major redesign.

1.3. Applications of Autonomous Car using ROS2

The development of autonomous car systems using Robot Operating System 2 has resulted in their widespread use across various fields because these systems can function independently without requiring human assistance. The following projects demonstrate how autonomous vehicle technology functions in real-time.

- Agriculture Automation: Autonomous robots can execute different farming tasks which include tracking crop development and applying pesticides and gathering harvested crops. The

implementation of these systems enables agricultural businesses to reduce their need for manual labor while they improve their productivity during large-scale farming operations.

- Self-Driving Vehicles: The advanced transportation systems that control self-driving cars permit these vehicles to navigate urban environments while avoiding obstacles and reaching their destinations without human assistance which improves road safety and traffic management.

- Warehouse Automation: Robots with navigation abilities operate in warehouses to move products while they improve operational performance and reduce staff needs for e-commerce and manufacturing operations.

- Search and Rescue Operations: The autonomous vehicles in search and rescue operations can operate in disaster areas because they help locate survivors and deliver essential supplies to places beyond human reach.

- Defense and Surveillance: UAVs are also used at borders for surveillance and reconnaissance missions and for dangerous premises via their monitoring activities.

- Healthcare Assistance: Robots operate independently throughout hospitals because they transport medical supplies, guide patients while helping medical staff with their routine tasks.

- Smart Cities and Urban Mobility: Autonomous vehicles enhance smart city transportation systems because they use their advanced traffic control systems to reduce traffic jams and build better transportation networks.

1.4. Importance of Autonomous Car using ROS2

Autonomous car systems developed with Robot Operating System 2 create substantial advancements for both contemporary robotics and self-driving vehicle technology. The systems use integrated sensing and computation capabilities together with decision-making systems to permit vehicles to drive themselves without needing human drivers as follows:

- **Reduction in Human Effort:** Self-operating cars decrease the demand for continuous human control. In fact, functions such as navigation monitoring and transportation can be done automatically.

- **Improved Efficiency:** optimized path planning together with real-time decision-making creates a high degree of efficiency for autonomous systems because it minimizes both time requirements and energy usage and operational interruptions.

- **Enhanced Safety:** Autonomous vehicles achieve safer navigation through their ability to monitor their surroundings continuously with LiDAR sensors and their capability to detect obstacles in real time.

- **Scalability in Applications:** Functionally, the ROS2-based architecture allows flexibility and therefore extension for different applications such as agriculture, logistics, and surveillance with little or no redesign.

- **Real-Time Decision Making:** The vehicle uses Simultaneous Localization and Mapping and adaptive localization methods to make intelligent decisions through real-time assessment of environmental conditions.

- **Cost Effectiveness in Long Term:** The design needs initial financial backing but automated systems lead to lower operational expenses through reduced workforce needs and improved efficiency.

- **Flexibility and Adaptability:** Based on robust perception and high efficiency in map-making, the performance of such intelligent systems is characterized by rapid adaptability, dynamic interaction, and a capacity to operate in different environments.

1.5 Advantages of Autonomous Car using ROS2

The Robot Operating System 2 Autonomous car systems developed through its technology bring three main benefits that boost both performance and operational capacity and system dependability for actual robot purposes. The system delivers main benefits to its users.

- **Modular and Reusable Architecture:** The modular system of ROS2 allows developers to build their system through the creation of separate independent components.

- **Real-Time Performance:** The ROS2 framework supports fast and efficient communication which enables the system to process sensor data while reacting to environmental changes without delay.

- **Accurate Mapping and Localization:** The system uses Simultaneous Localization and Mapping together with AMCL to create reliable maps which allow exact location tracking in unknown locations.

- **Dynamic Obstacle Avoidance:** The Navigation2 (Nav2) stack enables the robot to detect obstacles and re-plan paths dynamically which ensures safe navigation in changing environments.

- **Simulation-Based Development:** Developers can create and test their systems through virtual environments which use Gazebo and RViz because these tools provide virtual environment testing capabilities.

- **Reduced Human Intervention:** The system achieves its operational goals through autonomous navigation which allows it to function without human assistance while maintaining high efficiency.

1.6 Summary

Autonomous vehicle systems which operate on Robot Operating System 2 complete their driving functions through their sensors and processing power and control systems. The system utilizes LiDAR sensors together with its advanced algorithms to create environmental maps which assist in its real-time decision-making process. The vehicle uses Simultaneous Localization and

CHAPTER – 2
LITERATURE REVIEW

2.1. Intro to survey

The literature survey provides a complete examination of all research and published materials which exist in a particular field of study. The study investigates previous research on robotic navigation methods which include sensor integration and mapping techniques and control system development that are used in autonomous vehicle systems which operate with Robot Operating System 2. The survey aims to present current research findings while mapping key technological advancements which exist in autonomous navigation systems. Literature surveys are essential for several reasons:

- **Identification of Research Gaps:** The literature survey assists in discovering existing research limitations through identification of mapping accuracy issues and localization performance problems and real-time navigation limitations which need to be solved for further research to proceed.
- **Avoidance of Redundancy:** The study of earlier research about Simultaneous Localization and Mapping and ROS-based navigation systems enables researchers to bypass identical solutions which already exist.
- **Critical Evaluation of Existing Methods:** The system enables researchers to assess various algorithms through GMapping and Hector SLAM and SLAM Toolbox which helps them evaluate the performance of the algorithms across different operational settings.
- **Selection of Appropriate Methodology:** The analysis of existing research enables researchers to identify optimal tools and frameworks and algorithms which they will use in their projects including the selection of ROS2 to achieve enhanced scalability and real-time system capabilities.
- **Foundation for Further Research:** The literature survey establishes a fundamental base which enables researchers to create more advanced autonomous systems through built research and solution development of present system flaws.

2.2. Related Work

The researchers study published research which exists before their work about autonomous navigation and robotic systems and intelligent vehicles. The existing methods are studied to find their weaknesses which help in choosing the right methods for building the system. The field of autonomous vehicles that use Robot Operating System 2 has been studied through multiple research projects which examine SLAM algorithms and sensor integration and navigation frameworks.

2.1.1. Related work -1

Title: ROS-Based Autonomous Navigation Using SLAM

Authors: R. K. Megalingam, C. R. Nair, S. S. Suresh, A. Raj

Robots need autonomous navigation systems because humans face difficulty controlling robots in certain environments. The researchers created an autonomous navigation system which operates on the ROS platform and utilizes LiDAR sensors together with Simultaneous Localization and Mapping methods. The system enables the robot to create a map of an unknown environment while simultaneously estimating its position within that map.

The robot uses SLAM algorithms to create an occupancy grid map by processing real-time sensor data which it collects. The navigation system uses this map to plan a path from the starting position to the target location while avoiding obstacles. The implementation demonstrates effective integration of mapping, localization, and path planning using ROS framework.

The system enables real-time mapping together with full autonomous navigation capabilities for unknown environments. The system enables effective obstacle detection which helps in efficient path planning. The system needs accurate sensor data to operate correctly which limits its performance in environments that experience rapid changes or contain unpredictable elements.

2.1.2. Related work -2

Title: The Navigation2 (Nav2) Stack for ROS2

Authors: Steve Macenski, Francisco Martin, Ruffin White, Jonatan Clavero

To successfully navigate real-world environments autonomous navigation systems need two core functions which include path planning and obstacle avoidance. The Navigation2 (Nav2) stack provides a robust navigation solution which operates on the Robot Operating System 2 framework. The system enables users to build custom navigation solutions through its modular framework which manages global pathfinding and local obstacle detection and recovery functions.

The Nav2 system develops optimal movement paths by processing environmental maps together with current sensor measurements to determine paths from starting points to destination points. The system creates fresh robot path updates through continuous monitoring of changing environmental conditions and new obstacle detection. Robots can navigate safely without crashes through the systems which use cost maps to control their movements and use planners to determine their routes.

The main advantage of this system is its high modularity and flexibility, allowing customization for different robotic applications. The system enables both dynamic re-planning operations and real-time navigation functions for its users. The system needs complex setup procedures which involve detailed configuration methods together with parametric system adjustments and additional computational resources for its functions.

2.1.3 Related work – 3

Title: Hector SLAM for Real-Time Mapping

Authors: S. Kohlbrecher, O. von Stryk, J. Meyer, U. Klingauf

Autonomous robots need to map unknown spaces because it represents an essential requirement for their operation. The GMapping algorithm uses particle filter techniques to perform simultaneous localization and mapping through its implementation. The system generates occupancy grid maps by combining sensor data and robot motion information. The algorithm uses laser scan data and odometry to estimate the robot's position while building a map of the environment. It is widely used for indoor mapping due to its ability to produce accurate grid-based representations. The main advantage of this system is its reliability and accuracy in structured environments. The system has become a standard tool which researchers use for

testing their robotic systems. The system requires high processing power which makes it unsuitable for use in environments that involve extensive space or rapid changes.

2.1.4 Related work – 4

Title: SLAM Toolbox for ROS2

Authors: Steve Macenski

Accurate mapping and localization technologies serve as fundamental requirements for autonomous navigation systems. The SLAM Toolbox serves as a contemporary SLAM solution through its development for ROS2-based systems. The system offers two mapping methods which include synchronous mapping and asynchronous mapping to fulfill different implementation requirements. The system builds accurate occupancy grid maps through LiDAR and odometry data while it tracks the robot's movements in real time. The system enables ongoing mapping by allowing users to extend current maps throughout different time periods.

The main advantage of this system is its compatibility with ROS2 and its ability to handle large-scale environments efficiently. The system delivers high mapping accuracy which users can customize according to their mapping requirements. The system requires accurate parameter adjustments because its operational efficiency depends on the quality of its sensors.

2.3 Challenges

The development of autonomous vehicle systems through the Robot Operating System 2 creation process faces multiple technical issues because their systems need to execute real-time perception and mapping and navigation tasks during dynamic environment conditions when complex problems arise.

- **Privacy Sensor Accuracy and Noise:** Autonomous systems use LiDAR sensors for environmental perception which causes mapping and navigation challenges because the sensor data includes environmental condition-based disturbances that produce incorrect and imprecise data.

- **Localization Errors:** The process of navigation needs precise location data. The Simultaneous Localization and Mapping method experiences position estimation errors because its system drift condition develops when its odometry system encounters cumulative errors over time.

- **Dynamic Environment Handling:** Real-world environments experience ongoing changes because people and vehicles create moving obstacles. Automated systems encounter significant difficulties when they need to detect and respond to real-time environmental alteratio

Real-world environments experience ongoing changes because people and vehicles create moving obstacles. Automated systems face major difficulties when they have to detect and react to changes in their surroundings that occur without warning.

- **Computational Complexity:** The system needs to spend major computational power for processing large sensor data volumes which are essential for mapping and localization and path planning. The system experiences resource-related challenges that prevent it from functioning in a real-time manner.

- **Path Planning in Complex Environments:** The process of establishing an optimal and collision-free route through spaces that contain obstacles and restricted zones and undiscovered areas presents substantial challenges which demand advanced planning techniques.

- **System Integration:** System integration requires accurate management of different system elements which include sensors and control systems and SLAM technology and navigation systems.

- **Parameter Tuning and Optimization:** The SLAM Toolbox and Nav2 algorithms require their parameters to receive precise adjustments because this process enables their maximum operational performance. The system experiences navigation problems and mapping errors when users set the parameters to improper values.

- **Simulation to Real-World Gap:** The Gap between Simulation and Real-World Testing The testing of simulation systems shows different results between their successful execution in Gazebo testing and their actual testing because their operational methods do not match real-world physics and sensor performance.

2.4 Summary

The literature review on autonomous car systems using Robot Operating System 2 examines how modern robotic systems use mapping and localization and navigation methods. The researchers

investigated multiple studies to determine current research progress in autonomous navigation which includes sensor integration and SLAM algorithms and path planning frameworks.

The review shows that autonomous navigation depends on precise environmental detection from sensors which include LiDAR. Robots use Simultaneous Localization and Mapping techniques to create maps of new spaces while they track their own movements. The different SLAM methods GMapping Hector SLAM and SLAM Toolbox provide users with three distinct accuracy levels and computing efficiency and environmental adaptability.

The research demonstrates how the Nav2 stack navigation system functions within ROS2 to provide users with real-time capability for path creation and obstacle detection and dynamic plan alterations. The autonomous movement systems succeed in their primary function but face three major obstacles which include sensor interference and positioning mistakes and high computational demands.

CHAPTER – 3
SYSTEM DESIGN

3.1 Problem Definition

The Robots need to traverse various environments because they must execute their autonomous systems which require continuous obstacle detection and adaptability to environmental shifts. Robots which depend on manual control and pre-programmed maps can only operate in environments which their operators have previously defined. The process of autonomous navigation encounters difficulties because accurate localization needs to be achieved together with simultaneous mapping. The Simultaneous Localization and Mapping techniques enable robots to create maps while determining their current location, but these methods encounter challenges which include sensor noise and odometry drift and high computational requirements. The system needs to resolve two main problems for successful operation which include realtime path planning and obstacle avoidance. The robot should use its sensor data to identify obstacles while it modifies its path to achieve secure movement through the environment. The current systems might face difficulties when they attempt to operate in environments where objects move suddenly between points. Engineers encounter multiple challenges when they attempt to create a unified system which combines various elements like sensors and control systems and navigation algorithms. The Robot Operating System 2 system enables modular system design, but the development of SLAM and localization and navigation modules needs special system design work to connect these different components.

The current frameworks which operate with autonomous systems demonstrate their boundaries because these systems maintain static environmental requirements and they generate suboptimal routes and they lack capacity for instant environmental updates. A robust autonomous navigation system needs to be developed which can execute accurate mapping and dependable localization together with efficient pathfinding in real time using the capabilities of ROS2.

3.2 Proposed Methodology

The proposed system presents the design and implementation of an autonomous mobile vehicle using Robot Operating System 2 for real-time navigation. The system enables robot movement

through unknown environments by combining perception and mapping and localization and path planning. The robot starts its operation by using a 2D LiDAR sensor to gather environmental data through continuous scanning which produces distance measurements. The sensor data together with odometry information enables Simultaneous Localization and Mapping techniques to construct a two-dimensional occupancy grid map of the environment. The system uses Adaptive Monte Carlo Localization (AMCL) to determine robot position through its generated map. The system uses Navigation2 (Nav2) stack to execute both path planning and obstacle avoidance procedures. The navigation module creates a path from the starting position to the goal location which it updates when obstacles become visible to the system. The robot model for the entire system operates in a simulation environment which runs on Gazebo and uses URDF for robot model definition. RViz provides tools for visualizing sensor data and maps and tracking robot movements. The modular structure of ROS2 allows different system components which include sensors controllers and navigation algorithms to exchange information with high efficiency. The proposed system presents the design and implementation of an autonomous mobile vehicle using Robot Operating System 2 for real-time navigation. The system enables robots to navigate through uncharted territories by integrating three core functions which include perception and mapping and localization and path planning capabilities. The robot starts its operation by using a 2D LiDAR sensor to gather environmental data through continuous scanning which produces distance measurements. The combination of sensor data and odometry information allows Simultaneous Localization and Mapping techniques to develop a two-dimensional occupancy grid map of the surrounding area. The system uses Adaptive Monte Carlo Localization (AMCL) to determine robot position through its generated map. The system uses Navigation2 (Nav2) stack to execute both path planning and obstacle avoidance procedures. The navigation module creates a path from the starting position to the goal location which it updates when obstacles become visible to the system. The robot model for the entire system operates in a simulation environment which runs on Gazebo and uses URDF for robot model definition. RViz provides tools for visualizing sensor data and maps and tracking robot movements. The modular structure of ROS2 allows different system components which include sensors controllers and navigation algorithms to exchange information with high efficiency.

3.2.1 System Architecture(check)

The autonomous navigation system consists of five main components: the robot model, sensors, SLAM module, localization module, and navigation module. The system operates according to this established workflow:

System Initialization: The Robot Operating System 2 environment serves as the execution platform for this phase. The system establishes all necessary nodes which include sensor drivers and control interfaces and communication modules.

Robot Setup: The robot model is defined using URDF, which describes the physical structure including links, joints, and sensors. The robot movement control system uses ROS2 control plugins to establish its robotic movement.

Sensor Data Acquisition: The robot collects environmental data using a 2D LiDAR sensor. The sensor scans its entire environment while it records laser scan data which serves as the foundation for mapping and obstacle detection.

Mapping: The system uses Simultaneous Localization and Mapping techniques to process its sensor data. The SLAM Toolbox generates a 2D occupancy grid map by combining LiDAR data and odometry information.

Localization: The localization module uses Adaptive Monte Carlo Localization (AMCL) to estimate the robot's position within the generated map. The system tracks the robot's current position by using data from its sensors to calculate its movement.

Path Planning: The Navigation2 (Nav2) stack computes an optimal path from the starting position to the goal position using the generated map. It considers obstacles and environmental constraints while planning the path.

Navigation Execution: The Navigation2 (Nav2) stack computes an optimal path from the starting position to the goal position using the generated map. The system uses environmental data and obstacle information to create a safe path through the area.

Goal Completion: The robot achieves its objectives through movement when it executes the navigation commands which it received. The system monitors all sensor data in real-time while it updates the current path whenever it detects new obstacles.

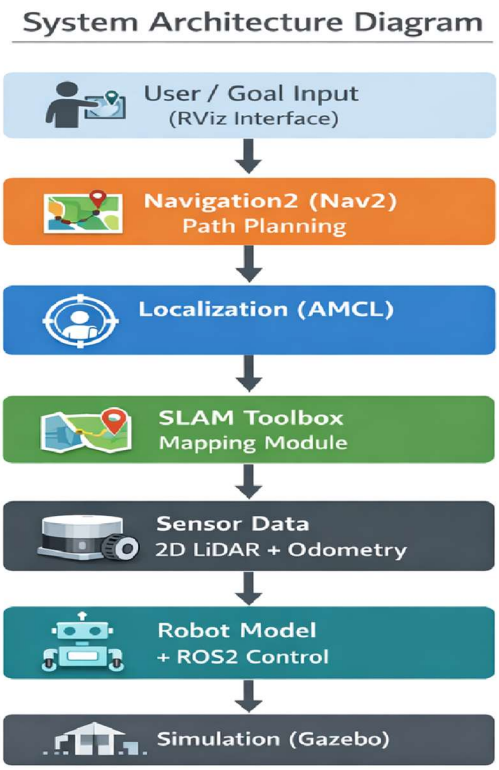


Fig 3.2.1 System Architecture

3.3 Modules

The system consists of the following main modules:

3.3.1 Sensor Module

Sensor Module The robot uses 2D LiDAR and wheel encoders as its sensors to gather environmental data which it receives in real time. The LiDAR continuously scans the surroundings and provides distance measurements, while odometry data helps track the robot's movement. This data is essential for mapping and obstacle detection.

3.3.2 SLAM Module

SLAM Module The system uses sensor data as input for Simultaneous Localization and Mapping processing in this module. The SLAM Toolbox generates a two-dimensional occupancy grid map of the environment by combining LiDAR data and odometry. Through this module the robot achieves understanding of its environment.

3.3.3 Localization Module

The system uses Adaptive Monte Carlo Localization (AMCL) to determine the robot's location in the map which it created. The system continuously tracks the robot's current position by using sensors to update its location throughout the navigation process.

3.3.4 Navigation Module

The Navigation2 (Nav2) stack in this module enables the robot to create and perform its movement plan. The system calculates the optimal path from the start position to the goal location and avoids obstacles dynamically. The robot executes the planned route by following control commands.

3.4 System Requirements Specifications (SRS)

The System Requirements Specification document establishes all functional and non-functional requirements for the autonomous vehicle system. The document describes system behavior and available system functions while explaining how the system operates and which functions it provides and how different system components work together in the Robot Operating System 2 framework. The system uses Adaptive Monte Carlo Localization (AMCL) to determine the robot's position on the generated map in this module. The system uses sensor data to track the robot's location and maintain precise tracking during its movement.

3.4.1 Requirement Analysis

Requirement Requirement Analysis serves as a software engineering process which connects two different stages of autonomous vehicle development. The purpose of Requirement

Analysis is to define system capabilities for execution and to determine system boundaries which will connect to various components including sensors and mapping and navigation systems. The system requirement analysis process begins when users provide their specifications. Users can present their required specifications through formal or informal methods. In formal method, the user clearly states the purpose of the autonomous system. The information provides developers with a strong foundation which they can use to conduct requirement analysis. The user does not explain the system purpose and expected results through informal method. The developer must discover system objectives and expected results through user conversations and analysis of navigation needs.

3.4.2 Software Requirements Specification (SRS)

The requirements specification document for an autonomous navigation system describes all system functions through use case descriptions which demonstrate how users will interact with the system. The system requirements document includes essential functional specifications together with essential non-functional requirements. Non-functional requirements impose constraints on the design or implementation. The software requirements specification document enlists all necessary requirements that are required for the project development. The system requirements are defined through detailed knowledge about the system requirements. The document originates from extensive discussions between the project team and system requirements analysis. A Software Requirements Specification minimizes the time and effort required by developers to achieve desired goals and also minimizes the development cost. A good SRS defines how an autonomous system will interact with sensors, hardware components, and users in a wide variety of real-world situations.

3.4.3 Functional Requirements User interfaces

The system needs to provide an interface which enables users to establish their target locations and view maps while tracking the robot's progress. The interface requires design to be intuitive while providing users with responsive access through RViz which enables them to interact with the system in real time.

User Management:

The system should provide functionalities suchas starting and stopping the robot, setting navigation goals, and controlling movement during mapping. The system provides operators with two control methods which include teleoperation for manual control and autonomous navigation capabilities for automated movement.

Audit Logs and Monitoring:

The system needs to provide monitoring tools which display robot activities and show sensor information and navigation progress. The system needs to display maps together with robot location information and laser scanning data and system communication details to assist users with their analysis and debugging tasks.

3.4.4 Non-Functional Requirements Performance:

The system requires real-time processing capabilities for sensor data together with mapping and navigation functions which need to deliver quick responses during robot movement. The system needs to achieve optimal performance through assessment of SLAM processing and path planning tasks and intermodule communication requirements.

Reliability:

The system must maintain its operational capacity across various environmental conditions while delivering accurate mapping and localization and navigation results. The system should operate correctly when faced with sensor noise and minor errors because these conditions do not require system shutdowns.

Scalability:

The system needs to enable new sensor implementation with algorithm and module integration without requiring substantial modifications to its current structure. The system provides capacity for future developments through new sensor implementation which can enhance system.

3.4.5 Software Requirements

.	**Software**	:	Robot Operating System 2 (Jazzy)
.	**Simulation Tool**	:	Gazebo Harmonic
.	**Visualization Tool**	:	Rviz
.	**Operating System**	:	Ubuntu

3.4.6 Hardware Requirements

.	**Hard Disk Drive**	:	128 GB
.	**Processor**	:	Dual core i5 (using pro and support pro)
·	**RAM**	:	8 GB

3.5 Introduction to UML (Unified Modeling Language)

The Unified Modeling Language exists as an essential modeling language which enables object-oriented system design and software engineering. The language serves software

engineering purposes yet provides a comprehensive vocabulary which enables modeling of system structure and application behavior and user interaction in robotic systems that operate autonomously. The system design visualization standard exists to standardize system design representation methods. Grady Booch and Ivar Jacobson and James Rumbaugh created the system at Rational Software between 1994 and 1995. The Object Management Group adopted the standard in 1997 and the organization has maintained it since then. The International Organization for Standardization (ISO) accepted the Unified Modeling Language as an official standard in the year 2000.

3.5.1 Class Diagram

Class diagrams exist as one of the most frequently utilized UML diagram types. The fundamental component of every object-oriented system exists as the class. The system displays its classes together with their attributes and operations and their class relationships. The system enables autonomous systems to display their different components which include sensors and controllers and navigation modules.

3.5.2 Component Diagram

The component diagram shows how system components relate to each other. The system components of the system require this method to handle their complex operations through multiple ROS2 nodes. The components of the system use interfaces to establish communication links which they connect through connectors.

3.5.3 Deployment Diagram

The deployment diagram displays both the system hardware components and the software applications that run on those components. The system enables autonomous systems to display their different components which include sensors and processing units and simulation environments.

3.5.4 Object Diagram

Object diagrams, which people sometimes call instance diagrams, share a strong resemblance with class diagrams. The diagrams display relationships between objects through authentic world scenarios while the system shows its state during a specific moment.

3.5.5 Use Case Diagram

Use case diagrams provide a graphical overview of the actors involved in a system, the functions performed by those actors, and their interactions. The system uses use case diagram to show how users interact with the autonomous robot system.

3.5.6 Activity Diagram

Activity diagrams present workflows through their graphical representation. The system can use them to explain functional workflows which include mapping processes and localization processes and navigation processes that operate in autonomous systems.

3.5.7 State Machine Diagram

State machine diagrams show how a system behaves in different states. The system uses these states to show robot states which include idle state and mapping state and navigating state and obstacle avoidance state..

3.5.8 Sequence Diagram

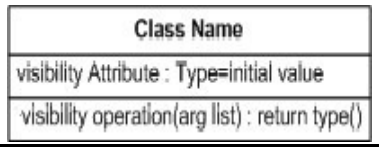
Sequence diagrams display how different system components work together through their communication sequence. The system uses these diagrams to show how ROS2 nodes communicate with each other during navigation processes.

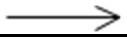
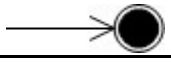
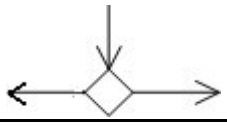
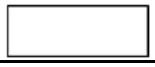
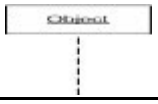
3.5.9 Collaboration Diagram

Collaboration diagrams function similarly to sequence diagrams but view component communication through exchanged messages.

3.5.10 List of UML Notations

Table 3.5.1 UML Notations

S. N0	SYMBOL NAME	SYMBOL	DESCRIPTION
1	Class		Classes represent a collection of similar entities grouped together.
2	Association		Association represents a static relationship between classes.
3	Aggregation		Aggregation is a form of association. It aggregates several classes into a single class.
4	Actor		Actors are the users of the system and other external entities that react with the system.
5	Use Case		A use case is an interaction between the system and the external environment.
6	Relation (Uses)		It is used for additional process communication.
7	Communication		It is the communication between various use cases.
8	State		It represents the state of a process. Each state goes through various flows.

9	Initial State		It represents the initial state of the object.
10	Final State		It represents the final state of the object.
11	Control Flow		It represents the various control flow between the states.
12	Decision Box		It represents the decision making process from a constraint.
13	Component		Components represent the physical components used in the system.
14	Node		Deployment diagrams use the nodes for representing physical modules, which is a collection of components.
15	Data Process/State		A circle in DFD represents a state or process which has been triggered due to some event or action.
16	External Entity		It represent any external entity such as keyboard, sensors etc.
17	Transition		It represents any communication that occurs between the processes.
18	Object Lifeline		Object lifelines represent the vertical dimension that objects communicate.
19	Message		It represents the messages exchanged.

3.6 UML Diagrams

Use Case Diagram

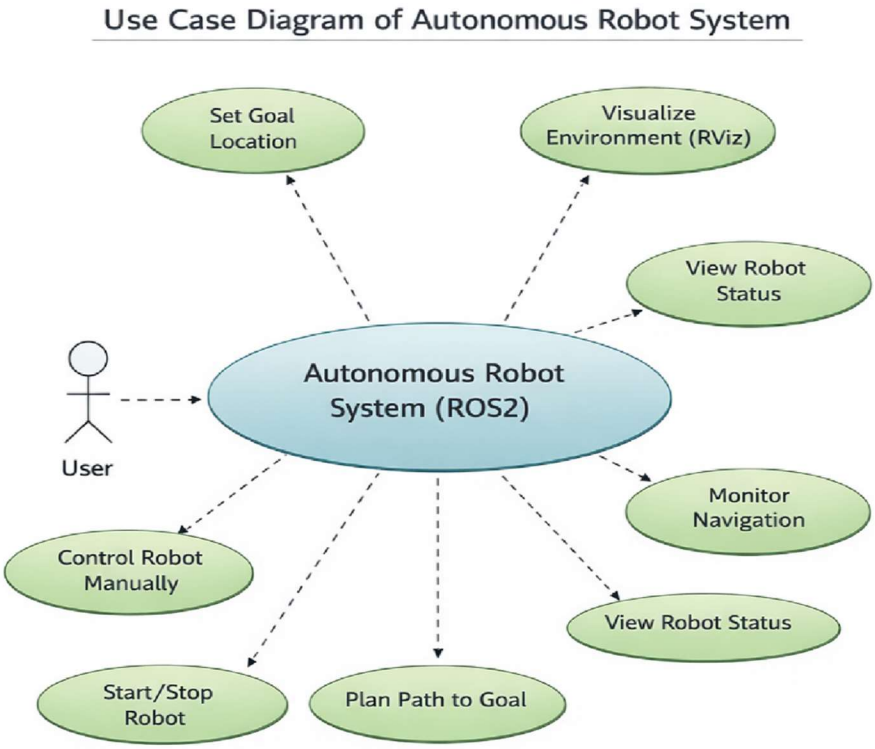


Fig 3.6.1 Use Case Diagram

Use case diagrams gives a graphic overview of the actors involved in a system, different functions performed by those actors and how these different functions are interacted. The actors involved in the above use case diagram are Data Owner, Delegator, Delegatee. The owner can register and login into the system using their credentials and then upload their health reports on the server in an encrypt ed format by requesting the key from delegatee. The delegator can register and login into the system and can view the health reports.

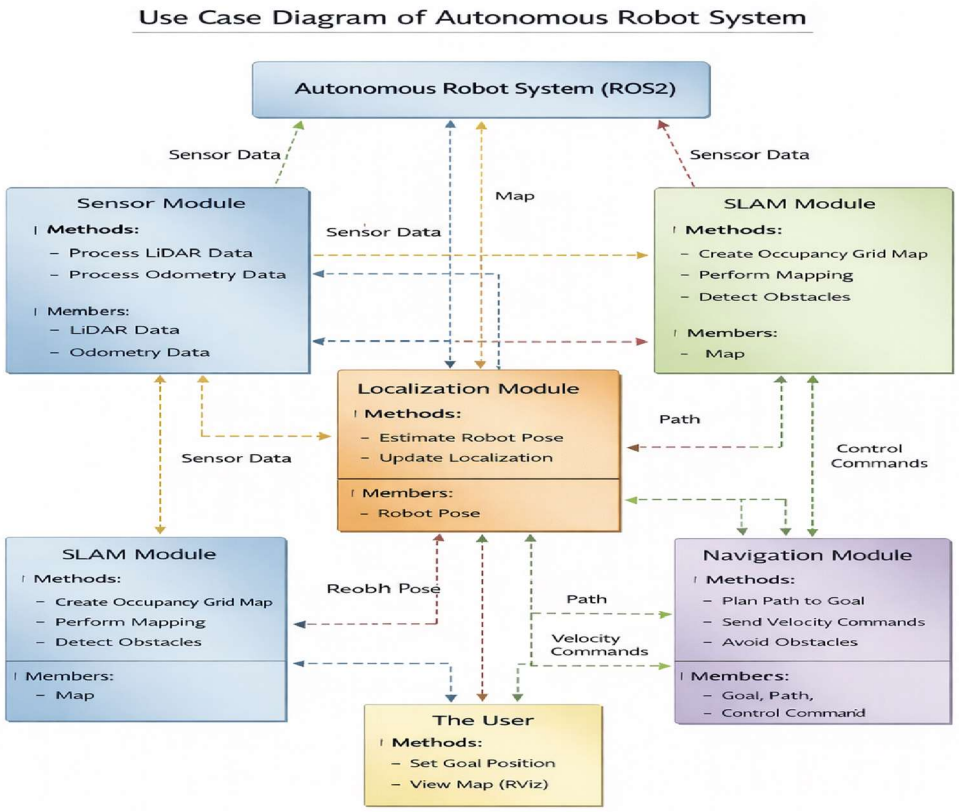


Fig 3.6.2 Class Diagram

The Class Diagram shows all the system classes. The system diagram displays its attributes together with its operational components. The above class diagram displays four classes which contain their respective attributes and operations. Most class diagrams consist of three main sections. The class name appears at the topmost section. The second section contains all the attributes that belong to that class. The final part contains all the operations that belong to that class.

Sequence Diagram

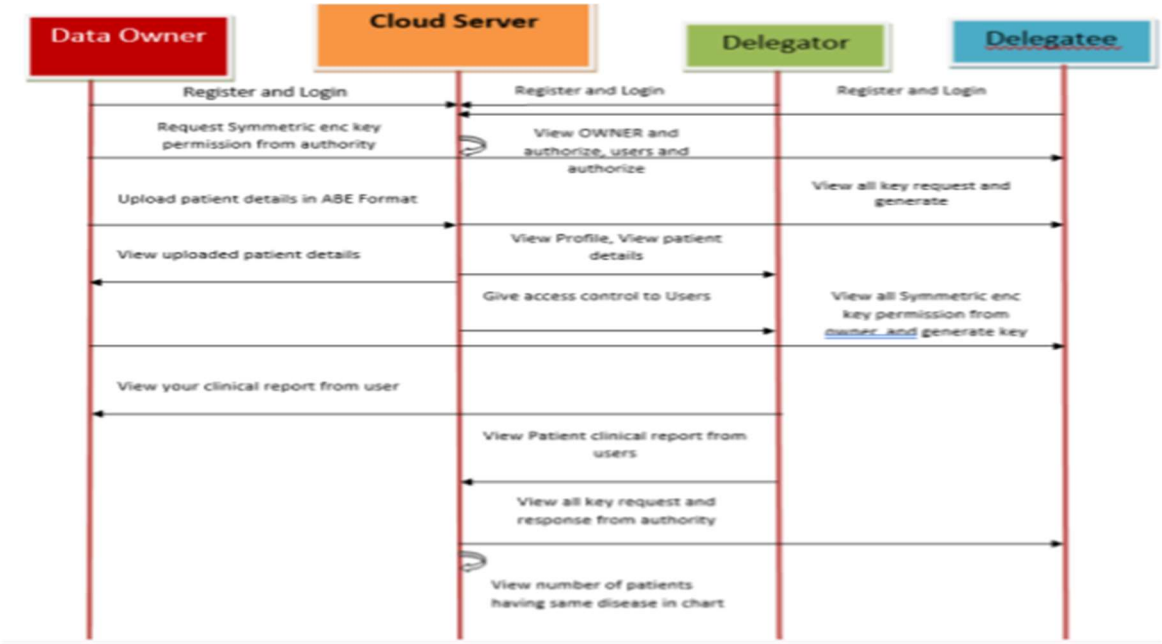


Fig 3.6.3 Sequence Diagram

A sequence diagram shows how objects interact with each other in a specific sequence that shows the timing of their interactions. The sequence diagrams show the operational sequence of system objects through their complete functional process. The system requires processes to handle message transmission between processes which need to perform their functions. The system development process uses sequence diagrams to show how use cases are implemented in the architectural view model of the system.

Data Flow Diagram

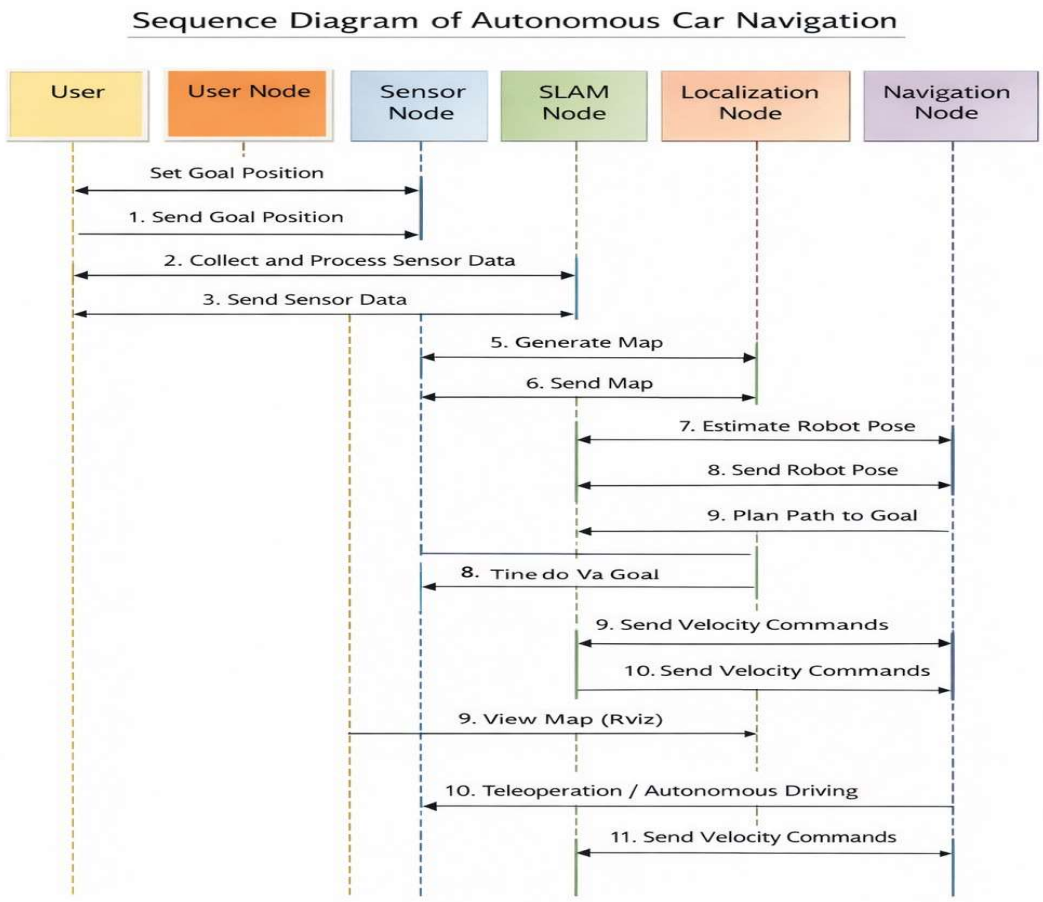


Fig 3.6.4 Data Flow Diagram

A data-flow diagram shows how information moves through a process or information system. The DFD shows all the input and output information that each entity and the entire process generate. A data-flow diagram has no control flow — there are no decision rules and no loops. A flowchart can show specific data operations that need to be performed.

Flow Chart Diagram

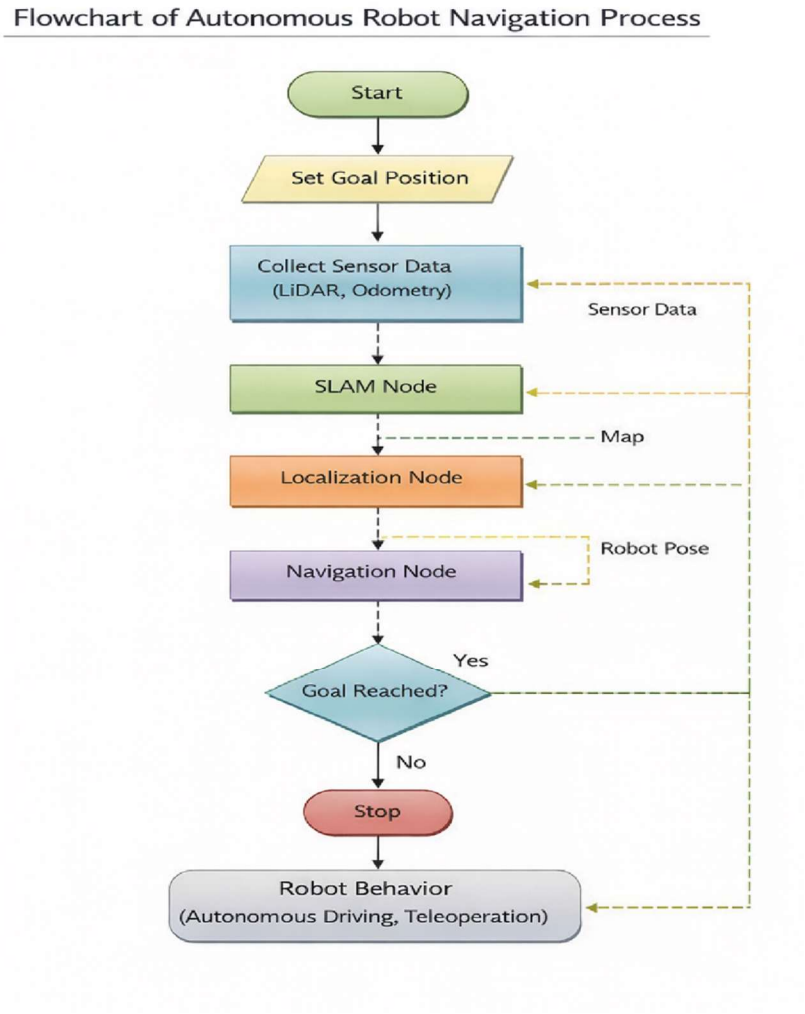


Fig 3.6.5 Flow Chart Diagram

A flowchart is a diagram that shows a process or system or computer algorithm. The method is commonly applied in various domains to create visual representations that enable people to document and study and plan and enhance and communicate complex procedures. Flowcharts, sometimes spelled as flow charts, use rectangles, ovals, diamonds and potentially numerous other shapes to define the type of step, along with connecting arrows to define flow and sequence.

CHAPTER – 4

SYSTEM IMPLEMENTATION

System implementation requires the establishment of all design specifications which define autonomous navigation system components including its physical and software elements. The system implementation process needs to guarantee system operational capability while meeting all necessary performance standards and quality requirements.

4.1 Technologies

ROS2 serves as a popular open-source robotics middleware that provides developers with tools and libraries and communication frameworks for their robotic application development.

4.1.1 ROS2

The system provides scalable development capabilities through its modular design which meets real-time operational requirements used in autonomous vehicle development. ROS2 operates through distributed systems which enable its various components called nodes to exchange information through three methods namely topics and services and actions. The system enables designers to combine sensors with control systems and navigation algorithms. The new system provides better real-time communication and security measures and performance upgrades when compared to the previous version. The main benefit of ROS2 permits users to operate the system on different platforms which include Linux and Windows and macOS operating systems. The system allows developers to use Python and C++ programming languages which gives them freedom to create applications. ROS2 enables autonomous systems to function through its support of communication between different system components which include sensor processing and Simultaneous Localization and Mapping and localization and navigation. The system provides a strong base which developers use to create scalable and efficient robotic systems.

4.1.2 Python

Python acts as a general programming language because it serves multiple purposes while maintaining high-level design. Robotics and automation fields widely implement Python because of its ability to create robotic applications through its simple and readable and flexible programming interface. The introduction of Robot Operating System 2 brought Python

programming to robotics through its framework, which enables developers to create Python-based node systems. Developers use Python to create scripts that manage robot operations and handle sensor information and execute navigation procedures. The main benefit of Python programming exists because developers can easily combine Python with different robotics libraries and tools. The system allows developers to create prototypes at high speed while achieving time savings over traditional development methods that use low-level programming languages. Python functions as a platform-independent software solution which operates on Linux and Windows and macOS operating systems. The project uses Python to create ROS2 nodes which manage sensor inputs and steer robot operations and connect all system components including Simultaneous Localization and Mapping and navigation. The system development process benefits from his straightforward design and efficient execution capabilities which make it perfect for creating autonomous systems.

4.1.3 Features of Python

Python stands as a programming language that people widely utilize because it operates as a high-level language which can serve multiple general purposes and its design promotes ease of use and code readability and its design allows users to create various applications:

- **Platform independence:** The Python programming language contains essential features which define its programming capabilities. Python exists as a cross-platform programming language because its codebase enables execution on various operating systems which include Windows and Linux and macOS without any required changes. This makes it suitable for developing cross-platform robotic applications.
- **Object-oriented programming (OOP):** Python enables developers to create modular reusable code through its support of object-oriented programming which includes encapsulation and inheritance and polymorphism and abstraction.
- **Simple and Readable Syntax:** Python provides developers with an easily understandable programming language because its syntax enables them to write programs using fewer code lines. This improves productivity and reduces development time.

- **Dynamic Typing:** Python implements dynamic typing which enables developers to create variables without needing to specify their exact data types. This method enables programmers to write code more efficiently while maintaining adaptable coding capabilities.
- **Support for scripting and automation:** Python serves as an excellent automation solution because it enables users to write scripts that control robot operations and handle sensor data and connect different components including Simultaneous Localization and Mapping systems.
- **Security:** Python offers multiple security solutions together with its libraries that enable encryption and authentication and secure data transmission. The framework is extensively utilized in robotic systems which require secure data management for their automated operations.
- **Portability:** Python enables developers to create robotic systems that work across different operating systems because it can operate on Windows and Linux and macOS without needing any system changes.
- **A large and active developer community:** Python has the advantage of great community support, where extensive documentation, libraries, and frameworks are available. This feature makes it easier during development, in case of developers using tools like Robot Operating System 2.

● **Presentational steadfastness:** Python interconnects versions well and expedites an environment where systems can be updated with relatively few changes. These are two of the main features that make Python the prime tool used in the making of distin- guished projects within various areas of robotics and autonomous systems.

- **Applications**
  - Robotics and Automation
  - Autonomous Systems
  - Artificial Intelligence and Machine Learning
  - Scientific and Data Analysis Applications
  - Web Development
  - Internet of Things (IOT)

4.1.4 Gazebo

Gazebo serves as an advanced simulation tool which enables users to create and evaluate robotic systems within virtual environments. The system provides accurate physical property simulation which includes gravity effects, collision dynamics, and sensor operation, thus enabling developers to create autonomous vehicles. Developers can use Robot Operating System 2 together with Gazebo which enables them to create robot simulations while testing navigation systems and measuring performance results without needing any physical equipment. The robot model is defined using URDF, and the environment can be customized to represent real-world scenarios. The project uses Gazebo to create an autonomous car simulation which produces LiDAR sensor data while testing the system's mapping and localization and navigation capabilities.

4.1.5 Features of Gazebo

Gazebo provides several important features for robotic simulation:

- **Realistic physics simulation:** Simulates real-world physics such as gravity, friction, and collisions for accurate robot behavior.
- **Sensor simulation:** Supports simulation of sensors like LiDAR, cameras, and IMU, enabling realistic perception data.
- **3D visualization:** Provides a graphical environment to visualize robot movement and interactions with surroundings.
- **ROS2 integration:** Works seamlessly with ROS2 for communication between simulation and control module.
- **Custom environment support:** Allows creation of different environments using models and custom objects.
- **Rich plugin support:** Gazebo provides a wide range of plugins for sensors, controllers, and physics engines, enabling customization of robot behavior and environment.
- **Integration with ROS2 architecture:** Gazebo works seamlessly with Robot Operating System 2, supporting modular design similar to distributed robotic systems.

- **Easy to learn and development:** Gazebo provides user-friendly tools and interfaces, making it easier to design, simulate, and test robotic systems without physical hardware.
- **Robust simulation environment:** Gazebo includes reliable simulation features such as collision detection, sensor accuracy modeling, and real-time updates, ensuring effective testing.
- Overall, Gazebo provides a powerful and flexible platform for simulating autonomous robotic systems, making it an essential tool for development and testing.

4.1.6 Steps for execution of ROS2

- The user sets a goal position using the RViz interface.
- The system sends the goal to the Navigation module through Robot Operating System 2.
- The sensor module collects real-time data from LiDAR and odometry
- The SLAM module processes the sensor data and generates a map of the environment.
- The localization module (AMCL) estimates the robot's current position within the map.
- The Navigation2 (Nav2) stack plans an optimal path from the current position to the goal.
- The system continuously monitors obstacles and updates the path dynamically.
- Control commands (velocity commands) are sent to the robot to execute movement.

4.1.7 RViz

RViz serves as a visualization tool within ROS2 which displays both robot data and sensor information along with environmental maps. The system offers a visual interface which enables users to observe the robot's current status and its ongoing activities during live operations. RViz enables users to display LiDAR scans and occupancy grid maps and the robot's current location and its intended movement routes. The system allows users to establish target locations while they watch the robot's progress toward those points.

In this project, RViz is used to:

- Visualize the map generated by SLAM
- Monitor robot movement and localization
- Set navigation goals

- Analyze sensor data and system behavior

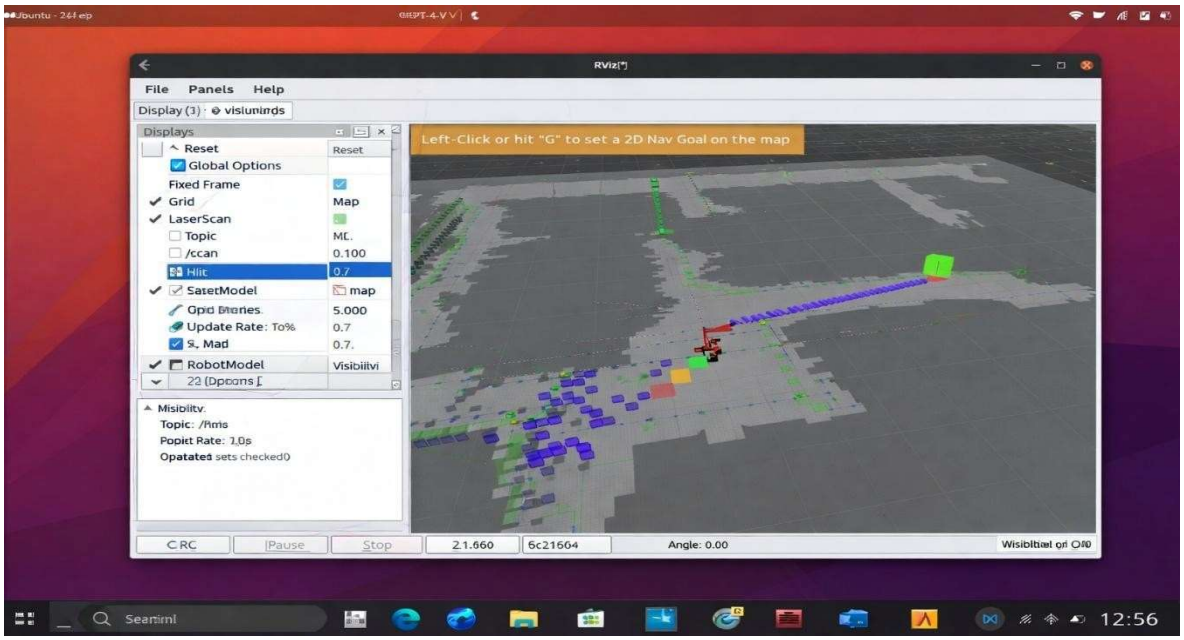


Fig 4.1 RViz

4.1.8 RViz Features

RViz serves as a comprehensive visualization tool which robotic systems use to present their current operational metrics and track their system performance. The software functions as a standard visualization tool which engineers use to display sensor information and mapping data and to track robot movements within the Robot Operating System 2 framework. The interactive interface of RViz enables users to view the robot's surrounding environment and its current location and its movement path. The system allows users to visualize multiple types of data which include LiDAR scans and occupancy grid maps and coordinate frames and robot models.

Some of the key features of RViz include:

- **Real-time visualization:**

  Displays live sensor data such as laser scans and robot movement.
- **Map visualization:**

  Shows occupancy grid maps generated using Simultaneous Localization and Mapping.
- **Robot model display:**

  Visualizes the robot structure defined using URDF.
- **Goal setting:**

Allows users to set navigation goals interactively.

- **Path visualization:**

  Displays planned paths and trajectories generated by the navigation system.

- **Debugging support:**

  Helps in analyzing system performance and detecting errors in mapping and navigation.

4.1.9 ROS2 Commands

- **ros2 launch my_diff my_sim.launch.py**

Launches the robot simulation environment in Gazebo, initializing the robot model, controllers, and basic system setup.

- **ros2 launch bot_slam online_async_launch.py**

Starts the SLAM module to perform real-time mapping using sensor data and generate the environment map.

- **ros2 run teleop_twist_keyboard teleop_twist_keyboard**

Enables manual control of the robot using keyboard inputs for movement and testing.

- **ros2 launch bot_nav localization_launch.py map:=./map.yaml use_sim_time:=true**

Initializes the localization module (AMCL) using the saved map to estimate the robot's position.navigation_launch.py

- **ros2 launch bot_nav navigation_launch.py**

Launches the Navigation2 (Nav2) stack to perform path planning, obstacle avoidance, and autonomous navigation.

4.2. Codes

navigation_launch.py

https://github.com/SaiPawanKummari/Autonomous-Car-using ROS2/blob/master/navigation_launch.py

localization_launch.py

https://github.com/SaiPawanKummari/Autonomous-Car-using-

ROS2/blob/master/localization_launc h.py

simln.launch.py

https://github.com/SaiPawanKummari/Autonomous-Car-using-ROS2/blob/master/simln.launch.py

online_async.launch.py

https://github.com/SaiPawanKummari/Autonomous-Car-using-ROS2/blob/master/online_async_launch.py

gazebo.launch.py

https://github.com/SaiPawanKummari/Autonomous-Car-using-ROS2/blob/master/online_async_launch.py

cartographer.launch.py

https://github.com/SaiPawanKummari/Autonomous-Car-using-ROS2/blob/master/cartographer.launch.py

CHAPTER – 5

SYSTEM TESTING

5.1 Test Case Description

The testing process identifies errors present in the autonomous navigation system. The system provides essential support to the robot's reliability and performance during its mapping and localization and navigation activities. Testing results lead to system improvements which enhance both accuracy and stability.

Psychology of Testing

The aim of testing is not only to demonstrate that the system works but also to identify conditions where it may fail. The project team conducts tests to find all existing problems which affect the processing of sensor data and the system's mapping capabilities and its localization and navigation functions. Testing uses system simulation in Gazebo to execute the system while RViz displays the system's output for error detection.

Testing Objectives

The main objective of testing is to identify errors in the autonomous navigation system systematically and efficiently. Formally, we can state:

- Testing is a process of executing the system with the intent of finding errors in mapping and navigation.
- A successful test is one that uncovers issues such as incorrect localization or path planning failure.
- A good test case is one that has a high probability of detecting errors in sensor data processing or obstacle avoidance.

Levels of Testing

The testing process needs to be conducted at multiple levels to identify all errors which exist in different system components.

System Testing

The testing process begins with system testing. The testing process exists to discover errors in software products. The testing team develops test cases to fulfill this

requirement. The testing team uses code testing as their method to perform system testing.

Code Testing

The autonomous navigation system testing method evaluates its logical operations. The method requires execution of multiple test scenarios to verify the functionality of each system module which includes sensor data processing and mapping and localization and navigation systems. The testing procedure verifies that all components operate correctly by checking their data flow processes and system performance. The testing process begins with individual module verification which leads to complete system testing after all modules pass their assessments. All modules undergo multiple testing phases to guarantee their operational correctness through various testing stages.

5.1 Types of Testing

- Unit Testing
- Integration Testing

Unit Testing

Unit testing verifies the functionality of separate system components. Testing detects errors in each module through system design and functional requirements. The project treats each ROS2 component as a separate module which includes the following components:

- Sensor module (LiDAR data)
- SLAM module
- Localization module
- Navigation module

Each module undergoes separate testing which uses distinct input sets. The system verifies sensor data accuracy while testing SLAM for correct map generation and testing localization for precise position estimation. The process confirms correct functioning of each module before starting system integration.

Integration Testing

The process of integration testing assesses how different system modules work together when they need to interact with each other. The primary concern is ensuring that all components communicate correctly within the Robot Operating System 2 framework.

The integration testing for this project needs to test three specific com ponents which include the sensor data connection to SLAM and the SLAM system's connection to localization and the data transmission between localization and navigation systems. The complete system undergoes testing to confirm its ability to operate all components in the correct way during autonomous navigation processes.

System Testing

The complete autonomous navigation system undergoes testing in this section. The system requirements serve as the benchmark for this process which aims to test whether the robot can execute its mapping and localization and navigation functions.

The entire system undergoes testing in the simulation environment through Gazebo while RViz provides monitoring capabilities. The testing process determines whether the robot can create a map and use Simultaneous Localization and Mapping to determine its location and reach the destination without hitting any obstacles.

Acceptance Testing

The testing process involves creating actual simulation scenarios which show that the autonomous system functions according to requirements. The testing process evaluates external robot functions which include its goal reaching and obstacle avoidance and path following abilities instead of examining its internal coding structure. The test cases assess various navigation scenarios which include testing under various conditions that involve obstacles and different levels of path complexity and various types of goal locations. The testing phase verifies that the system has achieved its goals which include creating precise maps and operating trustworthy location services and enabling secure navigation through Robot Operating System 2.

White Box Testing

This testing method evaluates all components of an autonomous system through individual module tests that search for potential system faults. The testing process confirms that all system functions work properly when all operational system paths have been tested. The testing method known as White box testing has an alternate name called Glass Box Testing. White box testing in this project tests the sensor data processing module and the mapping module and the

localization module and the navigation logic module. The execution paths in each module are tested through different test cases and sample inputs to confirm proper system operation.

Black Box Testing

This testing method evaluates each module as a complete unit through input output assessment while it avoids checking internal module operations. The module operates as a black box system that accepts input and generates output. The project uses black box testing to confirm system behavior through three test cases which include providing a goal position to the robot and evaluating its ability to reach the target and testing obstacle avoidance through obstacle introduction and assessing SLAM to check if the map generation process works correctly.

In this project, black box testing is used to validate system behavior such as:

- Providing a goal position → robot reaches the target
- Introducing obstacles → robot avoids them
- Running SLAM → map is generated correctly

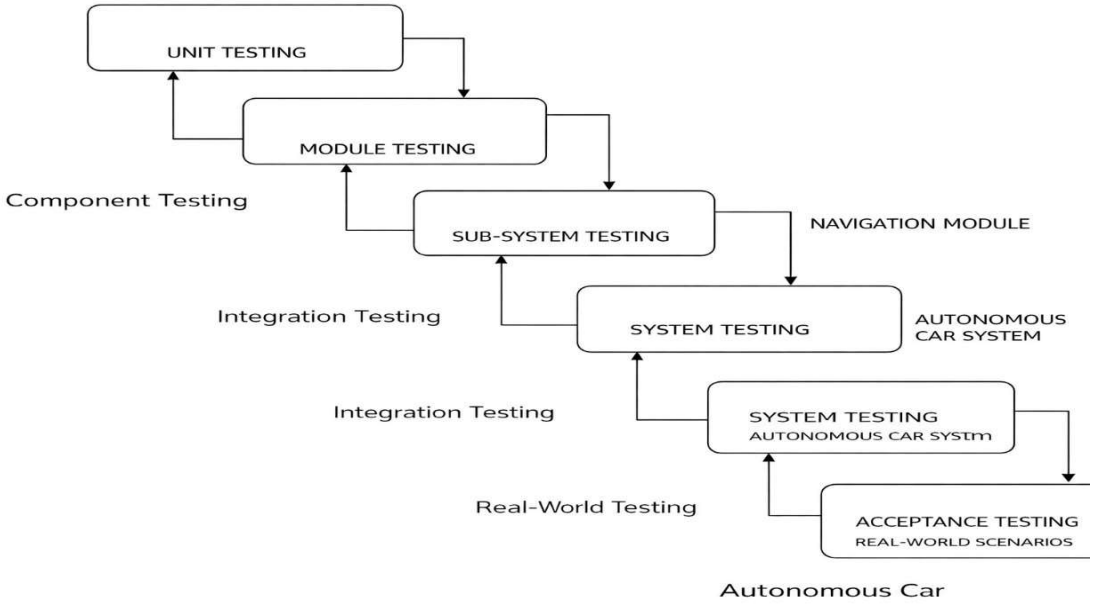


Fig 5.2.1 Testing Cycle

5.1 Test Cases (Autonomous Car using ROS2)

Positive Test Cases

Table 5.3.1 Positive Test Cases

Test Case	Input	Executed output	Actual Output	Status
Goal Setting	Set valid goal in RViz	Robot plans path	Path generated	Success
When data owner entered incorrect login credentials	Message “Incorrect User Name or Password” will be displayed	Message “Incorrect User Name or Password” is displayed	Successful	Success

Negative Test Cases

Table 5.3.2 Negative Test Cases

Function Name	Expected output	Executed output	Result status
When data owner is going to register without filling the necessary details.	Enter all the required details must be displayed.	Some error is displayed.	Failed
When data owner registers details in incorrect format.	Enter details in required format must be displayed.	Registers successfully	Failed

5.4 Results

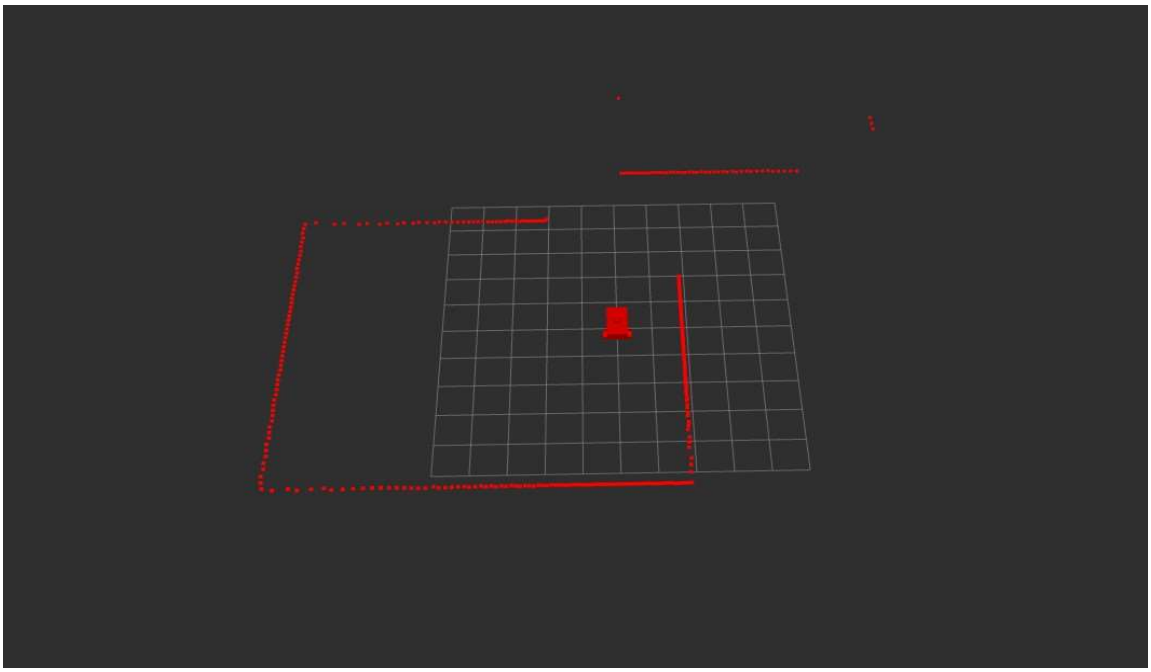


Fig 5.4.1 Mapping Environment

Gazebo Interface

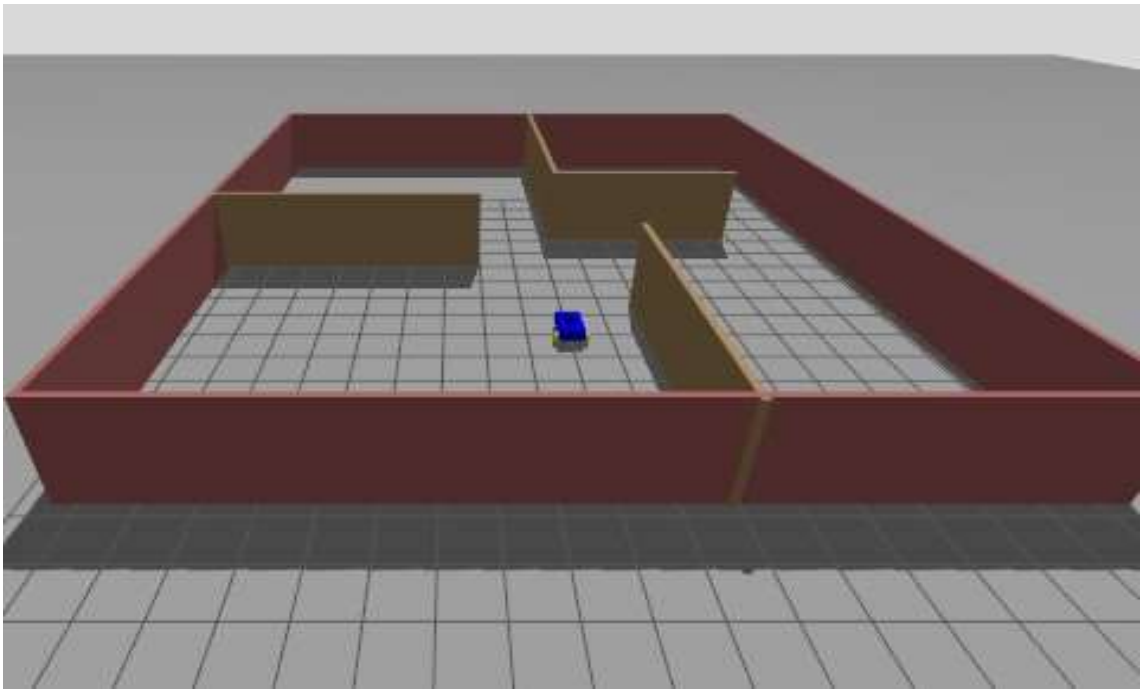


Fig 5.4.2 Gazebo Simulation

Mapped Environment

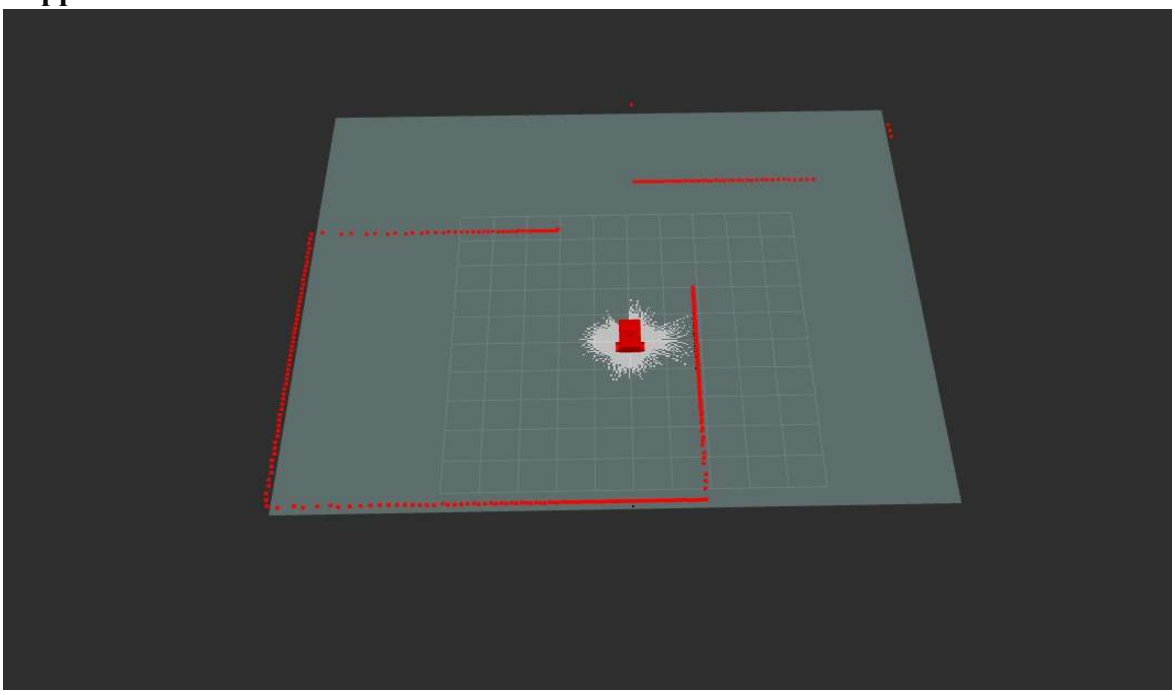


Fig 5.4.3 Mapped Environment using teleop key

Mapped Environment

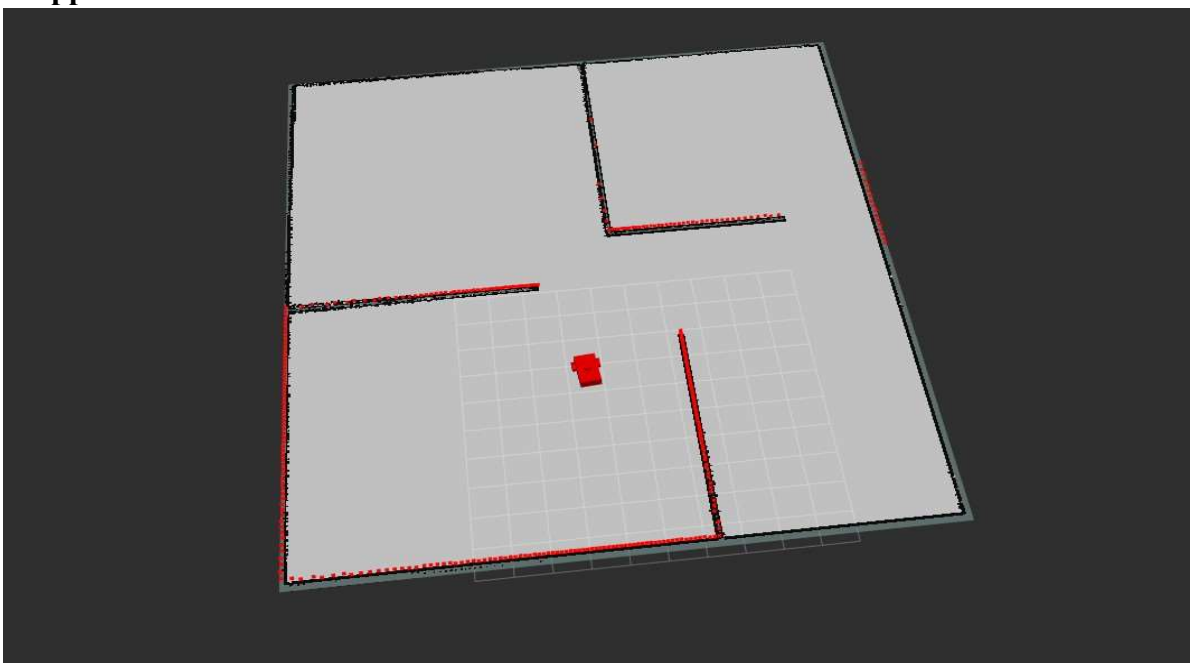


Fig 5.4.4 Mapped Environment

CHAPTER – 6

CONCLUSION

The Autonomous Car Using ROS2 project demonstrates how researchers combine intelligent systems with robotics middleware to create a dependable self-driving vehicle system. The system uses Robot Operating System 2 to establish effective communication between its different modules which include perception and planning and control elements. The project successfully integrates multiple sensors like cameras and LiDAR and ultrasonic sensors to understand the environment in real time. The vehicle uses advanced algorithms for object detection and localization and path planning to make safe decisions which allow it to drive without human help. The system's main strength comes from its ability to scale and use its modular design. The components of the system work together through ROS2 which enables that system to adopt new technologies through simple component upgrades. The system achieves real-time performance while adapting to changing conditions and maintaining higher reliability than standard methods. The system needs to solve three main problems which include managing complex traffic situations and enhancing sensor performance and establishing total system safety during unforeseen events. The system needs future improvements which will come from implementing advanced AI models and enhanced sensor fusion methods and conducting real-world testing across various environments. The project demonstrates how developers use ROS2 to create autonomous vehicle systems which deliver efficient performance and scalability, which creates better transportation solutions that enhance public safety.

CHAPTER – 7

FUTURE ENHANCEMENTS

The present system establishes a solid base for self-driving vehicles yet multiple segments need betterment and additional features to strengthen the system and bring it nearer to actual operational readiness.

1. Advanced AI Models: Object detection and lane detection and traffic sign recognition will achieve higher accuracy through the adoption of advanced deep learning models which will handle complex environments that include busy streets and dimly lit areas.

2. Improved Sensor Fusion: The system requires better data fusion capabilities which combine LiDAR and camera and radar and GPS information to enhance perception results while minimizing errors from single sensor limitations.

3. Real-Time Optimization: The system requires optimization for enhanced processing speed through GPU and edge AI device implementation which will deliver low latency and improved real-time decision-making capabilities.

4. Autonomous Decision Intelligence: The vehicle requires reinforcement learning or behavior-based algorithms for implementation because these systems enable the vehicle to acquire driving knowledge which leads to improved decision-making abilities during subsequent periods.

5. Enhanced Path Planning: The planning algorithms need upgrading to develop new algorithms which can manage dynamic obstacles and traffic signals and complex road conditions that involve intersections and roundabouts.

6. V2X Communication: The system needs to establish Vehicle-to-Everything communication which enables the vehicle to connect with other vehicles and traffic signals and infrastructure elements for enhanced safety and operational efficiency.

7. Cloud Integration: The system needs to establish a connection with cloud platforms which will enable data storage and remote monitoring and automatic model updates.

8. Simulation and Testing: The system requires advanced simulators CARLA Simulator and Gazebo to conduct extensive testing before the actual deployment in real-world environments.

9. Safety and Redundancy Mechanisms: The system needs to incorporate backup systems and fail-safe mechanisms which will enable it to handle sensor or system failures while maintaining operational reliability during critical conditions.

10. Real-World Deployment: The project requires hardware upgrades and improved calibration methods and safety standard compliance testing to achieve its goals of entering real-world testing.

CHAPTER – 8

REFERENCES

[1] M. Bojarski, D. Del Testa, D. Dworakowski, et al., "End to End Learning for Self-Driving Cars," NVIDIA, 2016.

[2] A. Dosovitskiy, G. Ros, F. Codevilla, et al., "CARLA: An Open Urban Driving Simulator," in *Proceedings of the 1st Annual Conference on Robot Learning*, pp. 1–16, 2017, CARLA Simulator.

[3] S. Thrun, W. Burgard, and D. Fox, "Probabilistic Robotics," MIT Press, 2005.

[4] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, "Introduction to Autonomous Mobile Robots," MIT Press, 2011.

[5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 779–788, 2016.

[6] M. Montemerlo, S. Thrun, D. Koller, and B. Wegbreit, "FastSLAM: A Factored Solution to the Simultaneous Localization and Mapping Problem," in *Proceedings of the AAAI National Conference on Artificial Intelligence*, pp. 593–598, 2002.

[7] Open Robotics, "ROS2: Design, Architecture, and Applications," 2019.

[8] J. Levinson, M. Montemerlo, and S. Thrun, "Map-Based Precision Vehicle Localization in Urban Environments," in *Proceedings of Robotics: Science and Systems*, 2007.

[9] H. Badue, R. Guidolini, R. Carneiro, et al., "Self-Driving Cars: A Survey," *Expert Systems with Applications*, vol. 165, 2021.

[10]IEEE, "Sensor Fusion for Autonomous Vehicles," *IEEE Transactions on Intelligent Transportation Systems*, various issues.

APPENDIX – A

PLAGIARISM REPORT

AUTONOMOUS CAR USING ROS2

by 23095A3308 KUMMARI SAI PAWAN

Submission date: 23-Apr-2026 01:26PM (UTC+0530)
Submission ID: 2941212149
File name: 23095A3308.pdf (1.5M)
Word count: 10219
Character count: 61896

23095A3308.pdf

ORIGINALITY REPORT

12%	10%	4%	9%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	Submitted to Middle East College of Information Technology Student Paper	2%
2	www.rgmcet.edu.in Internet Source	1%
3	Submitted to Vardhaman College of Engineering, Hyderabad Student Paper	1%
4	teresaproject.eu Internet Source	<1%
5	amostech.com Internet Source	<1%
6	Submitted to Yonsei University Student Paper	<1%
7	ftshivamtiwari.blogspot.com Internet Source	<1%
8	Submitted to Liverpool John Moores University Student Paper	<1%
9	slidetodoc.com Internet Source	<1%
10	Chuan Wu, Yaochen Li, Ling Li, Le Wang, Yuehu Liu. "Caption Generation from Road Images for Traffic Scene Construction", 2020 IEEE Intelligent Vehicles Symposium (IV), 2020 Publication	<1%
11	Submitted to University of Lincoln Student Paper	<1%